

IF 260

Sistem Operasi

Minggu 9

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

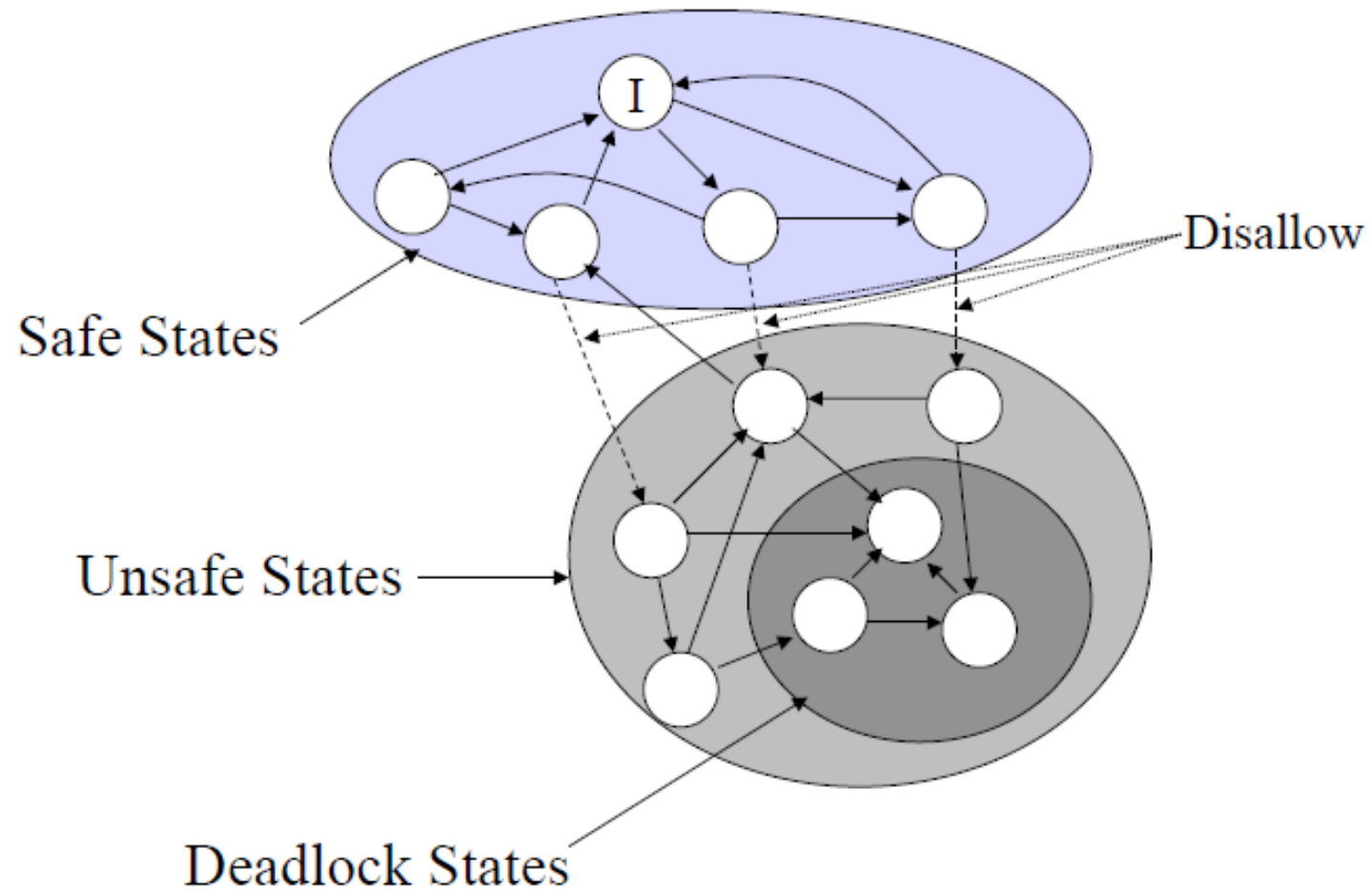
Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- **Review**
- **Topik Minggu 9:**
 - **Konsep Pengelolaan Memori (Legacy)**
 - **Virtual Memory (Modern)**
 - Paging
 - Segmentation

Review

Safe, Unsafe, Deadlock States



Safe & Unsafe State:

3 processes and 1 jenis resource

Total resource = 10

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3
(a)

	Has	Max
A	3	9
B	4	4
C	2	7

Free: 1
(b)

	Has	Max
A	3	9
B	0	—
C	2	7

Free: 5
(c)

	Has	Max
A	3	9
B	0	—
C	7	7

Free: 0
(d)

	Has	Max
A	3	9
B	0	—
C	0	—

Free: 7
(e)

(a) adalah safe state

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3
(a)

	Has	Max
A	4	9
B	2	4
C	2	7

Free: 2
(b)

	Has	Max
A	4	9
B	4	4
C	2	7

Free: 0
(c)

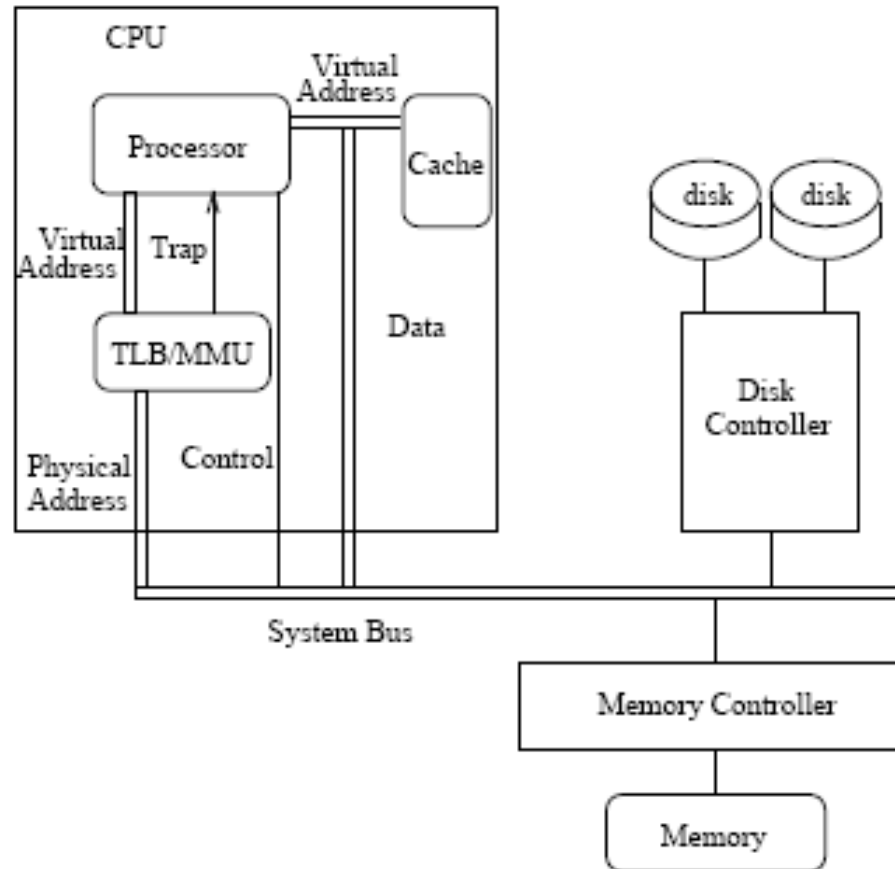
	Has	Max
A	4	9
B	—	—
C	2	7

Free: 4
(d)

(b) adalah unsafe state

Konsep Pengelolaan Memori

Latar Belakang



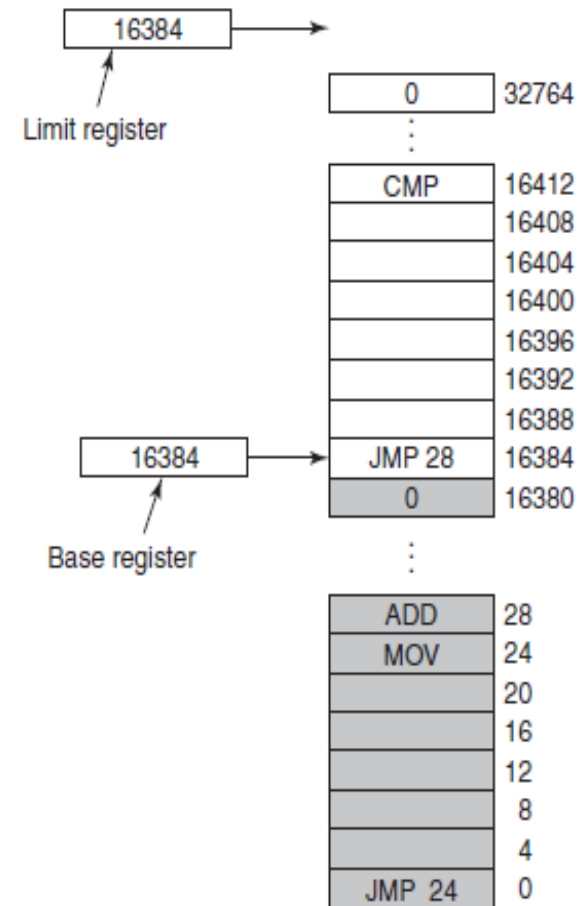
- Executable program disimpan di disk
- Saat executable program dijalankan, OS loads program (instruksi dan data) tersebut ke main memory, dan menjalankannya sebagai sebuah process

Memory Management Unit (MMU)

- Hardware device yang memetakan logical/virtual address ke physical address (alamat fisik)
- Program dari pengguna hanya tahu virtual/logical address, dan tidak perlu berhubungan langsung dengan physical address
- Tiap address (alamat) yang dirujuk oleh process dimanipulasi oleh MMU untuk mendapatkan physical address untuk kemudian dikirimkan ke lokasi fisik yang dirujuk
 - Contoh: penggunaan base dan limit register (untuk alokasi memori berurutan (contiguous))

Base & Limit Registers

- CPU menyertakan registers untuk alokasi memory, misal **base** dan **limit register**
- Ketika process berjalan, base register berisikan physical address untuk awal process, dan limit register berisikan panjangnya process
- Dengan cara seperti ini, bisa diatur agar setiap process memiliki private address space
- Catatan: untuk tiap memory reference, dilakukan operasi addition dan comparison



Memory Management

- **OS bertugas membuat abstraksi memori (memory abstraction) pada media penyimpanan (storage media) dan mengelolanya (manage)**
- **Abstraksi memory → address space**
 - **Physical address space: address space yang disediakan hardware**
 - **Logical/virtual address space: address space yang dilihat oleh process**
- **Tantangan:**
 - **Multiprogramming (beberapa program di memory)**
 - Memerlukan pengaturan lokasi dan proteksi program di memory
 - **Jumlah dan ukuran aplikasi yang bertambah besar dan besaran memory yang terbatas**
 - Memerlukan teknik pembagian memory, seperti partition, swapping, paging, segmentation
 - Penggunaan virtual memory sehingga media penyimpanan seperti disk, USB flashdisk dapat digunakan sebagai physical memory seperti halnya RAM

Address Binding

(Pemberian alamat pada program)

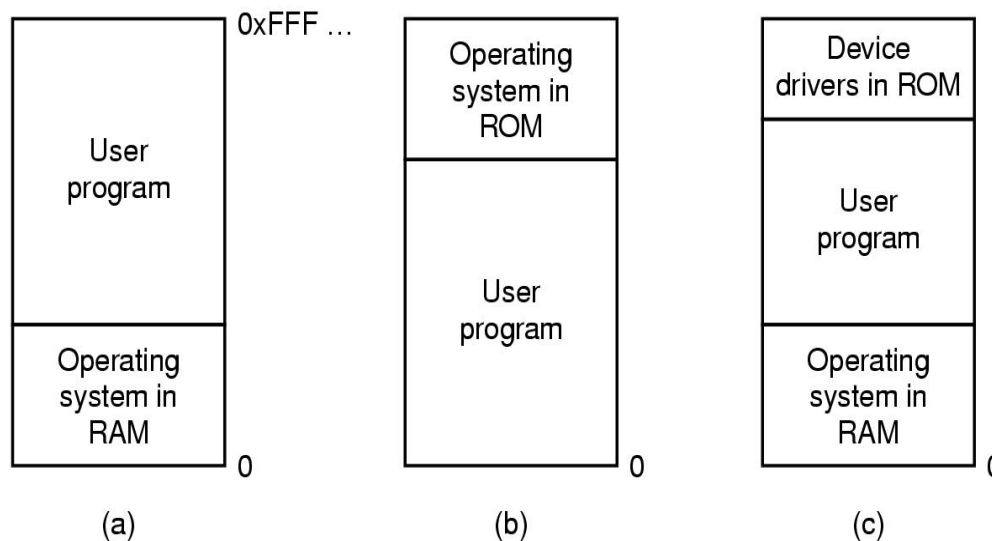
Untuk dapat dieksekusi oleh CPU, instruksi program dan data harus diberikan alamat yang dapat terjadi setelah:

- **Compiling**
 - Compiler mengubah source code ke object code (machine code yang berisikan instruksi dan alamat). Pengalamatan relatif terhadap program itu sendiri. Jika lokasi memory diketahui sebelumnya (contoh pada monoprogramming), maka absolute code dapat dibuat. Harus di-compile ulang apabila ada perubahan lokasi memory
- **Linking**
 - Linker menggabungkan beberapa object code, yang mana terjadi pengalamatan ulang untuk gabungan instruksi dan data dari tiap object code
- **Loading**
 - Alamat yang dibuat oleh compiler diatur ulang oleh OS yang disesuaikan dengan posisi awal address space dari process
- **Execution**
 - Pengalamatan ditunda sampai run-time di mana process dapat dipindahkan dari segment memory satu ke yang lain.

Monoprogramming:

Sudut pandang memory management

- Monoprogramming: hanya eksekusi satu user process pada satu waktu
 - OS mendapatkan address yang tetap (memory tertinggi untuk DOS)
 - User process dieksekusi pada segment-segment yang berturutan(contiguous section of memory)
 - Compiler data menghasilkan code dengan pengalamatan sesuai dengan physical address
 - Maximum address = memory size – OS size



Multiprogramming:

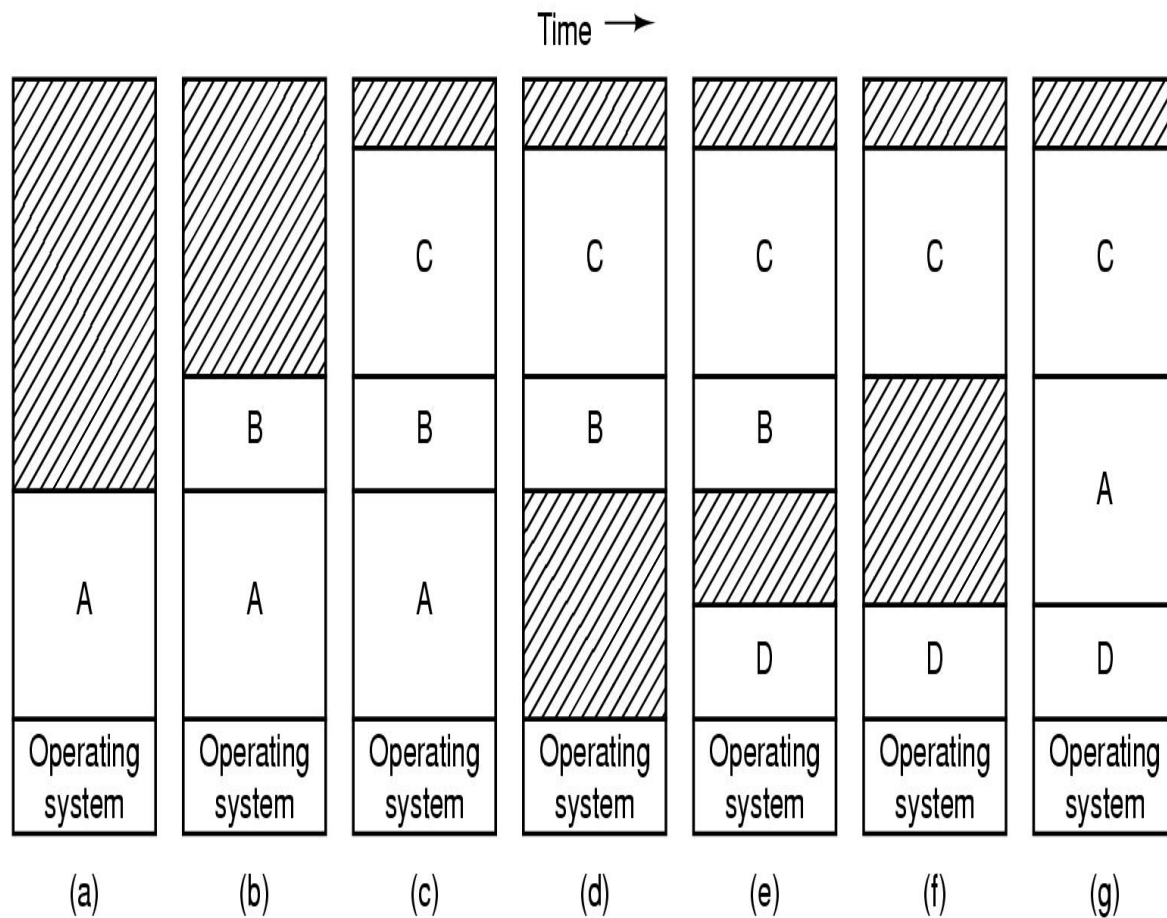
Sudut pandang memory management

- **Multiprogramming/Multitasking:** eksekusi beberapa user process secara bergantian (concurrent)
 - Beberapa process menggunakan memory secara bersamaan
 - Process tidak perlu tahu lokasi physical memory yang diberikan
 - OS bertugas memberikan perlindungan sehingga process yang satu tidak mengganggu address space process milik OS atau process yang lain
- **Relocation:** OS dapat menyesuaikan pengalamatan process disesuaikan dengan policy memory management yang digunakan
 - **Static relocation:** saat load time, OS menyesuaikan alamat dari process sesuai dengan posisinya di memory, dan tidak dapat diubah lagi
 - **Dynamic relocation:** disesuaikan secara dinamis sesuai kebutuhan

Multiprogramming: memory management

- Tanpa Virtual Memory → memuat program-program yang berukuran lebih kecil dari kapasitas physical memory
 - Fixed partition (dengan pemartisian tetap)
 - Dynamic partition (dengan pemartisian dinamis)
- Swapping → jika besaran RAM tidak mencukupi
 - Pindahkan process ke disk sehingga RAM dapat digunakan oleh process yang lain
- **Dengan Virtual Memory**
 - dapat memuat program-program yang berukuran lebih besar dari pada kapasitas physical memory.
 - Non-contiguous memory allocation (bagian dari process tidak perlu diletakkan pada posisi yang berurutan)
 - Paging
 - Segmentation
 - Segmentation dan paging

Swapping

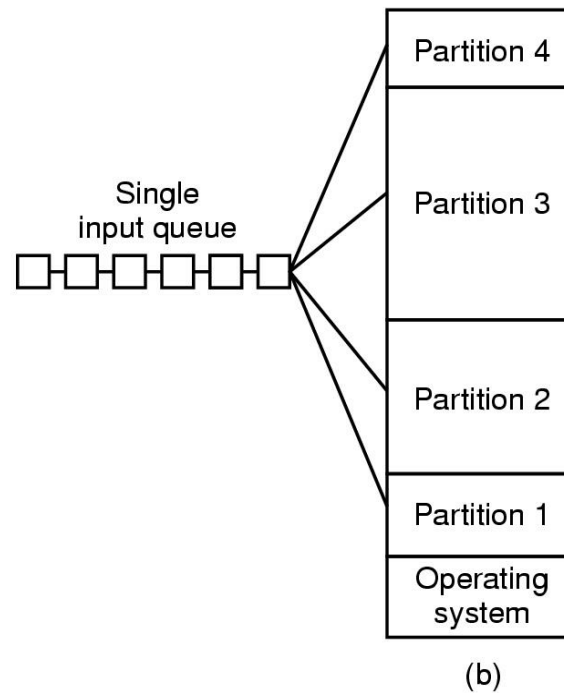
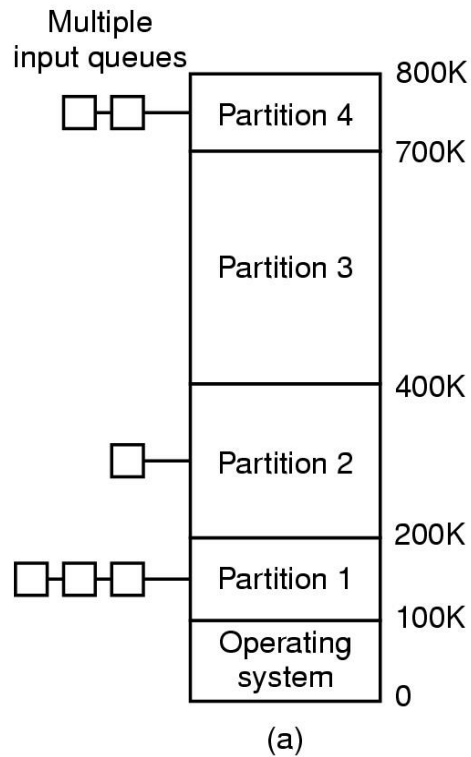


Fixed Partition

- **Partisi memori dengan ukuran tetap:**
 - **Ukuran sama (equal-size)**
 - **Placement algorithm yang digunakan sederhana: letakkan process pada partisi tersedia selama ukuran process lebih kecil dari ukuran partisi.**
 - **Utilisasi memory tidak bagus (internal fragmentation)**
 - **Ukuran tidak sama (unequal-size) :**
 - **Single dan multiple input queue**

Fixed Partition

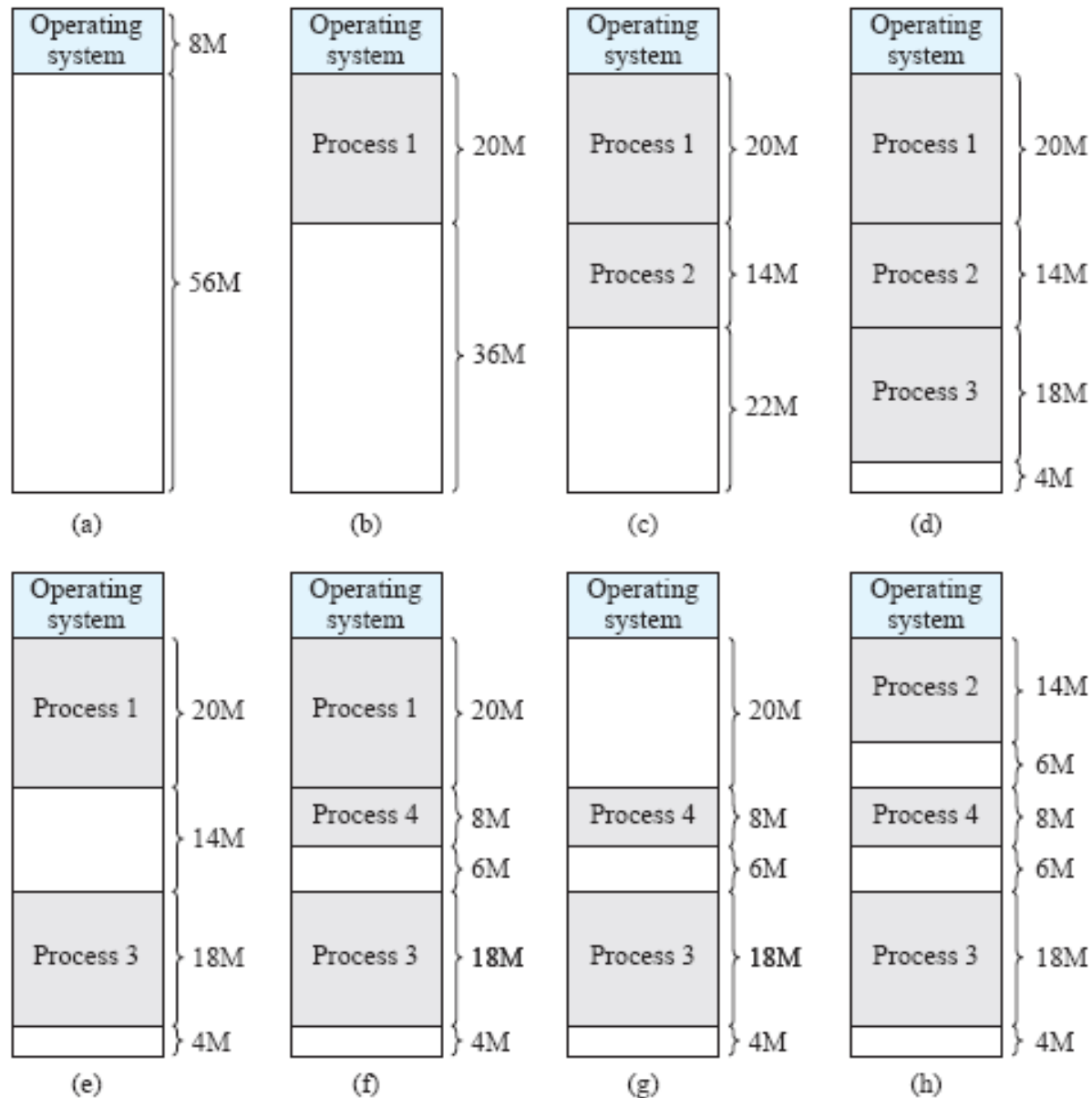
- Dengan ukuran tidak sama (unequal-size)
 - Single dan multiple input queue



Dynamic Partition

- Partisi memory jumlah dan ukuran yang dinamis
- Saat process dibawa ke memory, alokasikan partisi memory sesuai dengan ukuran process
- Problem yang dapat terjadi: external fragmentation (bagian memory di luar partisi menjadi semakin fragmented)
- Untuk mengatasi external fragmentation
 - Compaction
 - OS dari waktu ke waktu memindahkan process-process di memory sehingga mereka bersebelahan dan free memory menjadi satu blok.
 - Placement algorithm (algoritma untuk meletakkan process pada blok yang sesuai) → lihat di slide berikutnya

Ilustrasi Dynamic Partition



Dynamic Partition: Placement Algorithm

- **Best-fit:** memilih blok dengan ukuran yang paling mendekati permintaan
 - Pros: mengurangi blok memori yang terbuang
 - Cons: search keseluruhan blok memory yang kosong
- **First-fit:** scan memory dari awal dan pilih blok pertama yang ukurannya mencukupi permintaan
 - Pros: Cepat
 - Cons: Menyisakan blok-blok berukuran kecil di awal
- **Next-fit:** scan dari posisi peletakkan terakhir dan pilih blok pertama yang ukurannya mencukupi permintaan. Kinerja serupa dengan First-Fit, tapi perlu tambahan pointer untuk mengetahui posisi terakhir
- **Worst-fit:** memilih blok dengan ukuran yang paling besar dibandingkan dengan permintaan
 - Pros: mengurangi blok memory kosong yang berukuran kecil
 - Cons: search keseluruhan blok memory yang kosong, mengurangi blok memory kosong yang mungkin diperlukan oleh process yang berukuran besar

Virtual Memory (Memori Maya)

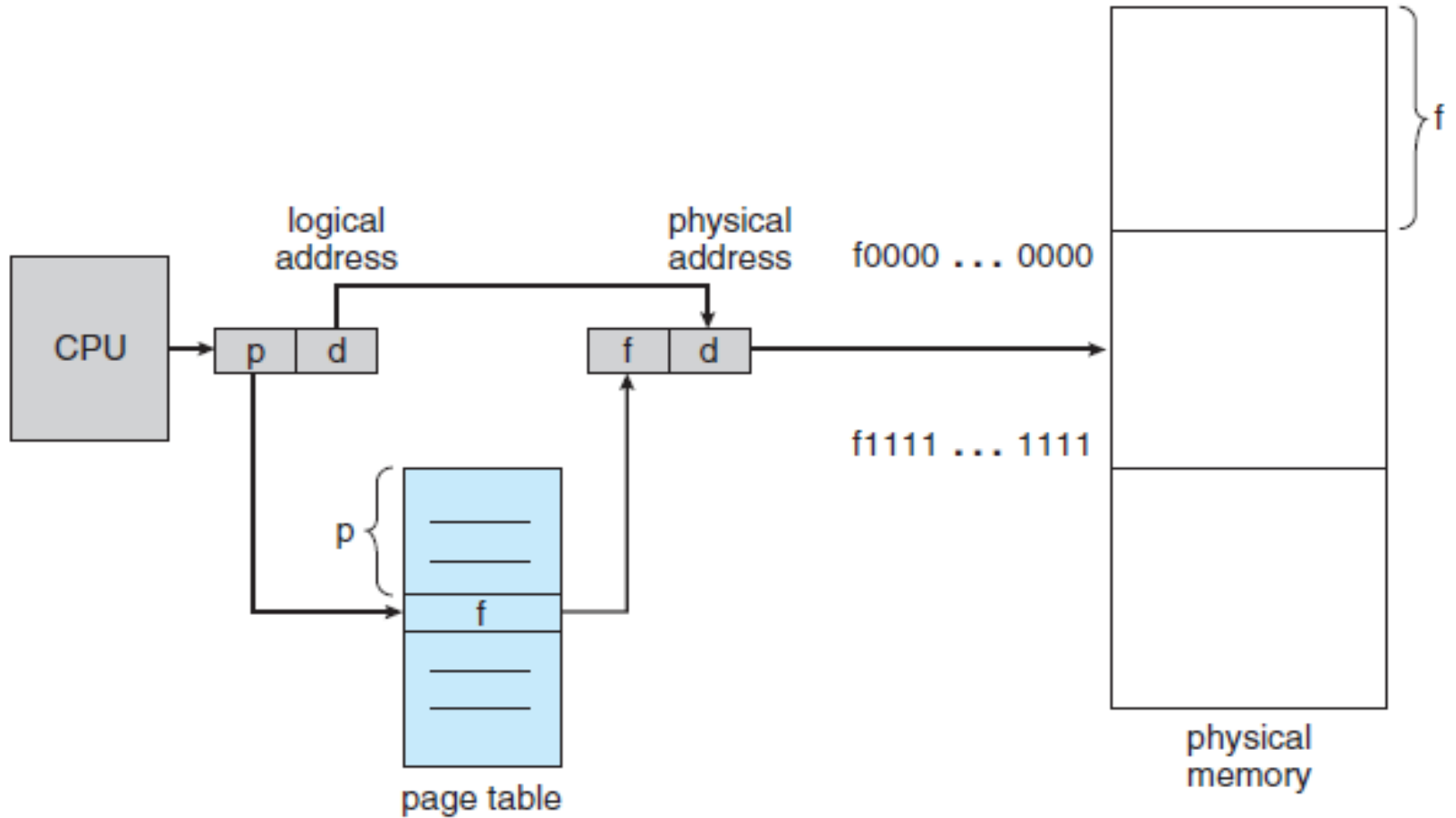
Konsep

- **Process umumnya tidak menggunakan keseluruhan ruang memory saat berjalan pada satu waktu**
 - Dengan konsep Virtual Memory, bagian-bagian yang diperlukan diletakkan di physical memory, dan yang lain di media penyimpanan lainnya (disk, dsb.)
- **Alokasi pada physical memori tidak harus berurutan (non-contiguous), dan juga transparan bagi pengguna**
- **Keuntungan:**
 - Meningkatkan efisiensi sistem multiprogramming
 - Dapat menjalankan process yang ukurannya melebihi kapasitas ruang pengalamatan (address space) di physical memory (memori utama fisik/RAM)
- **Virtual memory dapat dilakukan dengan:**
 - Paging
 - Segmentation
 - Kombinasi paging dan segmentation

Paging

- Bagi virtual address space atas unit-unit kecil yang berukuran tetap yang disebut dengan Page
- Bagi physical address space atas unit-unit kecil yang disebut Page Frame, dan ukurannya sama dengan Page
- MMU melakukan pemetaan Page ke Frame, dan Frame tidak harus contiguous (berurutan) pada physical memory
- Penggunaan Page ukuran kecil dan tetap menghilangkan external fragmentation dan mengurangi internal fragmentation (umumnya $1/2$ page untuk tiap process)

Paging



Paging: pemetaan

page 0
page 1
page 2
page 3

logical
memory

0	1
1	4
2	3
3	7

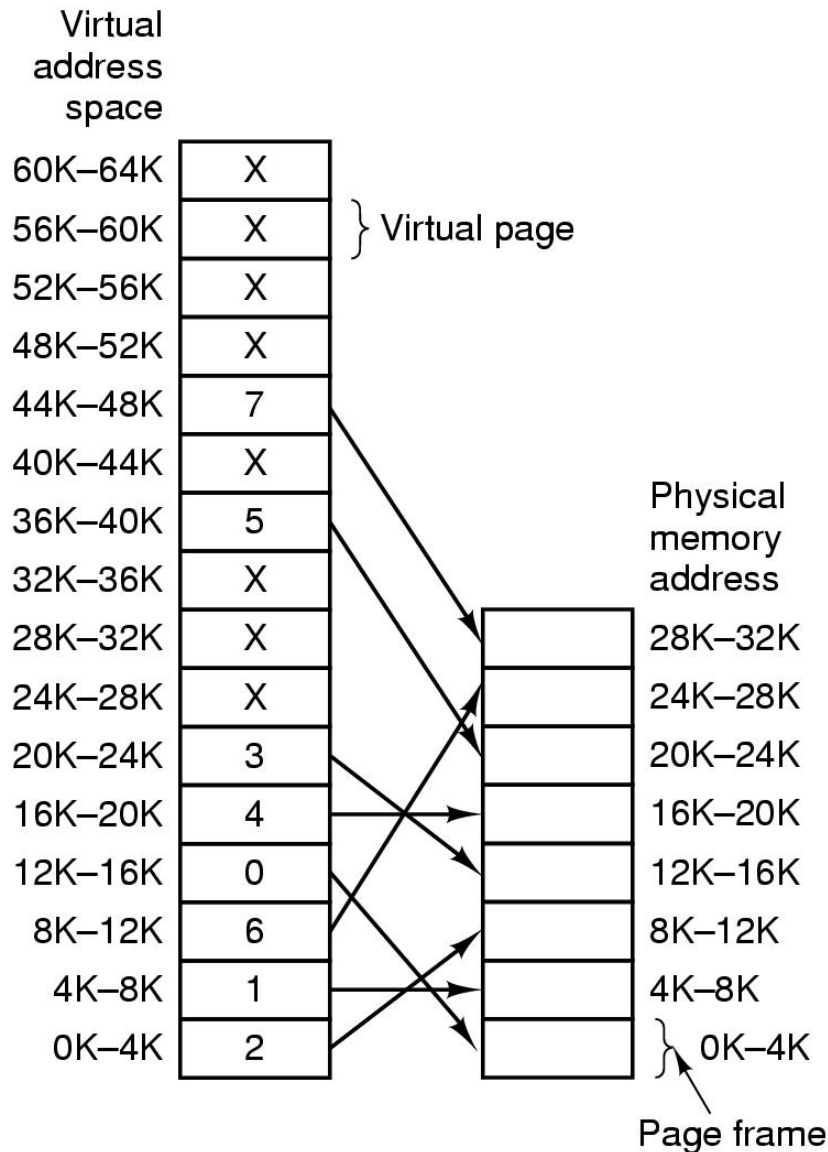
page table

frame
number

0	
1	page 0
2	
3	page 2
4	page 1
5	
6	
7	page 3

physical
memory

Contoh: Konsep Paging



16 bit address → 64KB Virtual Space

32 KB Physical Space

4 KB Page size

16 Pages

8 Frames

Contoh soal:

Virtual address 8400 = Physical address ?

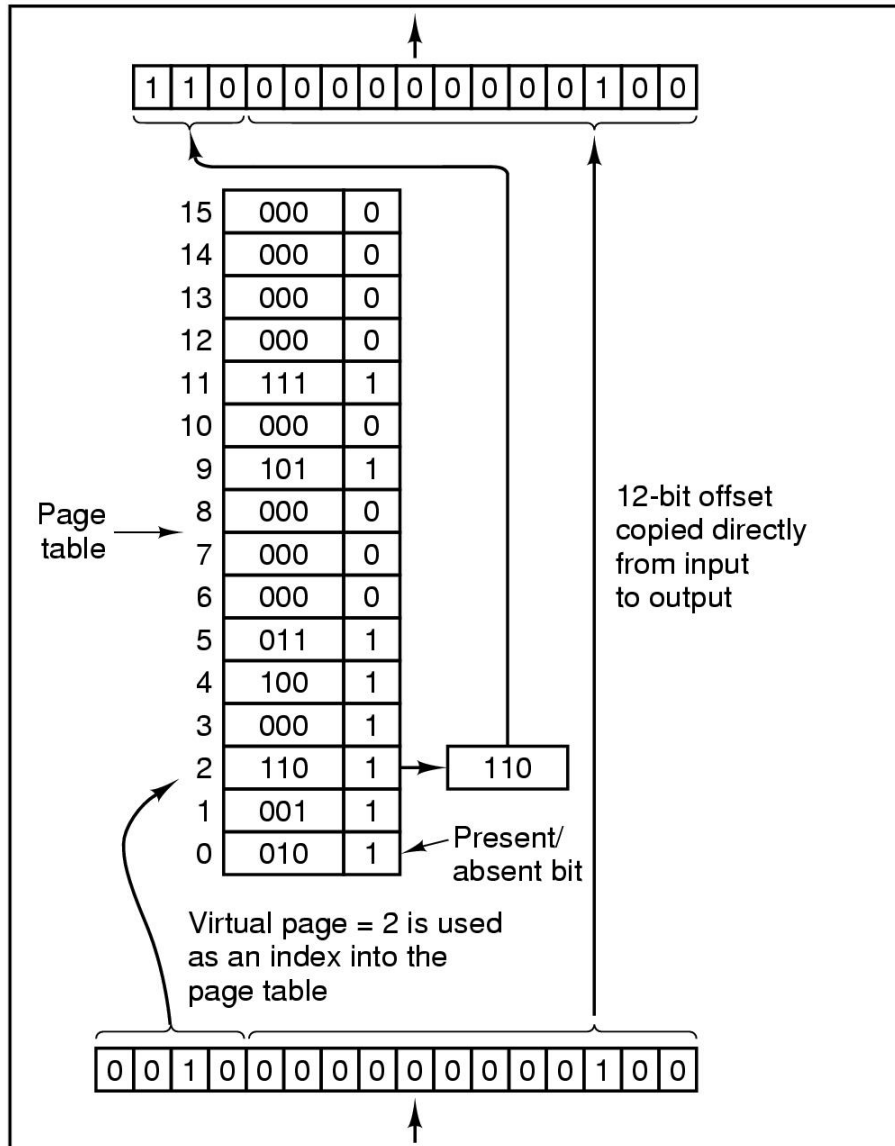
Virtual address 500 = Physical address ?

Virtual address 28300 = Physical address?

Apabila tiap page (decimal): 0 - 3999

Apabila tiap page (binary) : 0 - 4095

Contoh generik operasi internal pada MMU untuk 16 buah 4-KB Pages



Outgoing
physical
address
(24580)

Physical address:
Page frame number & offset

Virtual page number merujuk pada
Index dari page table
Page table dikelola oleh memory
manager (kernel)
Page table entry berisikan page frame
number

Virtual address:
Virtual page number & offset

Incoming
virtual
address
(8196)

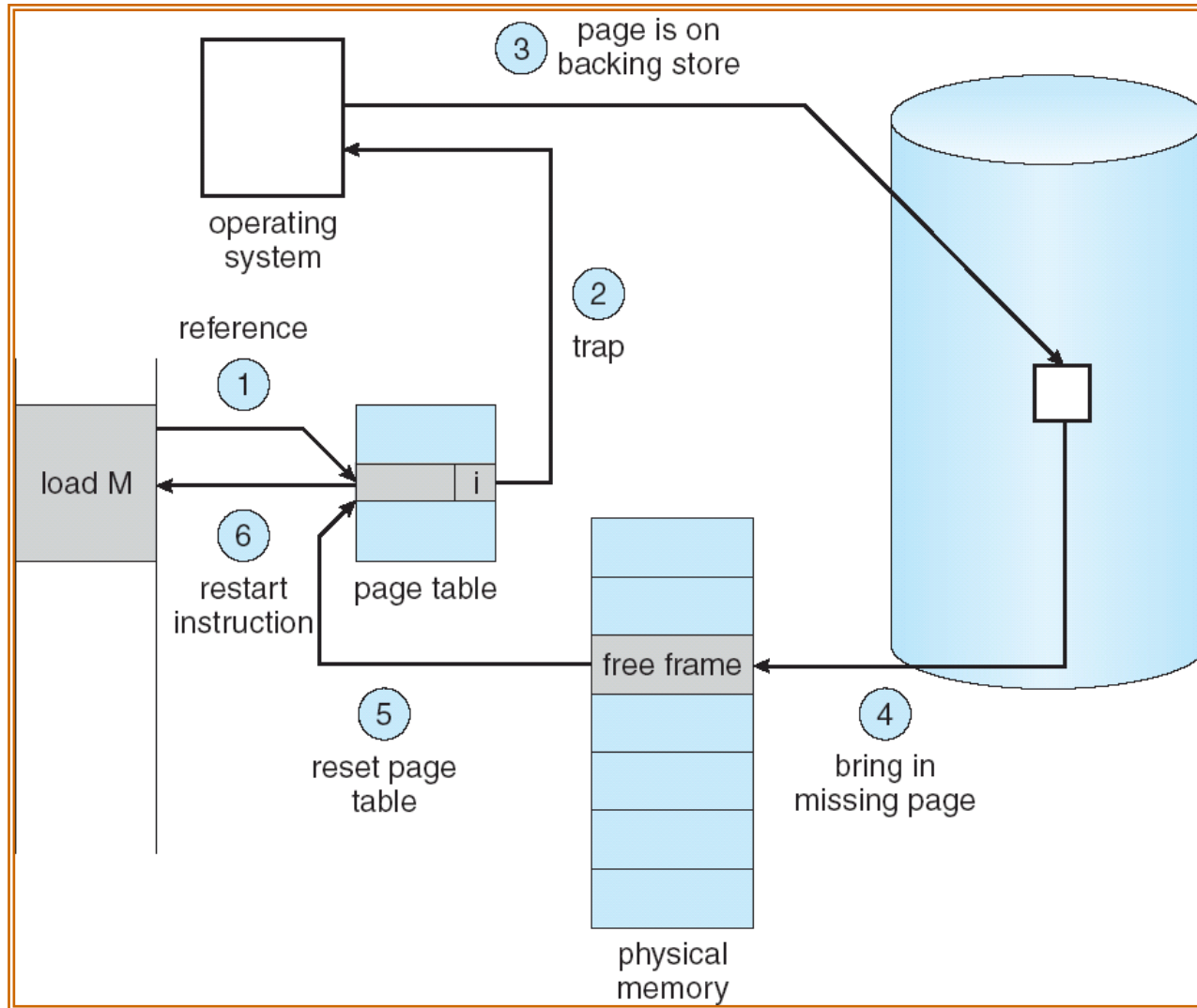
Reference String dan Page Fault

- **Reference string:** suatu urutan dari virtual page number yang mengacu kepada memory reference dari suatu program

Menggunakan gambaran di slide sebelumnya:

- **Apabila program melakukan memory reference secara berurutan dari 0 sampai 28K, maka**
 - **Page reference string: 0,1,2,3,4,5,6**
- **Saat Page 6 dirujuk, terlihat status bit =absent, yang artinya Frame yang di-referenced belum dipetakan di memory**
 - **Terjadi Page Fault**
 - **OS kemudian menjalankan suatu interrupt handler service atas interrupt akibat page fault ini, untuk membawa page yang ada di disk ke memori, kemudian update page table**

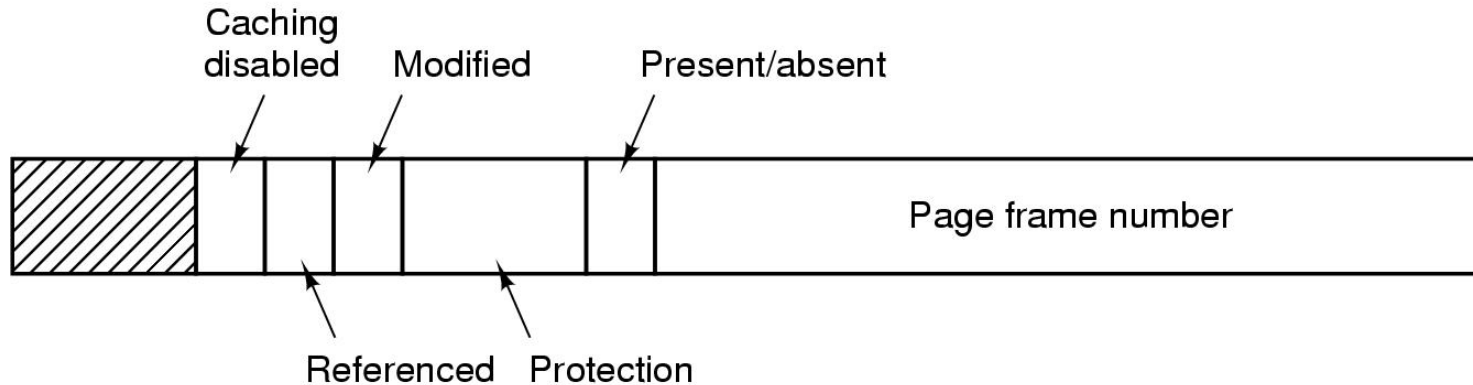
Apabila terjadi Page Fault



Contoh address translation menggunakan paging

- Sistem dengan 32-bit contiguous address space (ruang pengalamatan yang berurutan)
 - $2^{32} = 4 \text{ GB}$ (4.2950×10^9 bytes)
- Asumsi page size = 4Kb (4096 bytes)
 - Perlu $\log_2(4096) = 12$ bits
- Sisa 20 bit digunakan untuk menentukan penomoran pages
 - Terdapat 2^{20} PTE (Page Table Entries)
- Terjemahkan virtual address 0x12345678 ke physical address menggunakan paging di mana PTE(0x12345) berisikan 0x01000
 - Physical address = 0x01000678

Struktur generik Page Table

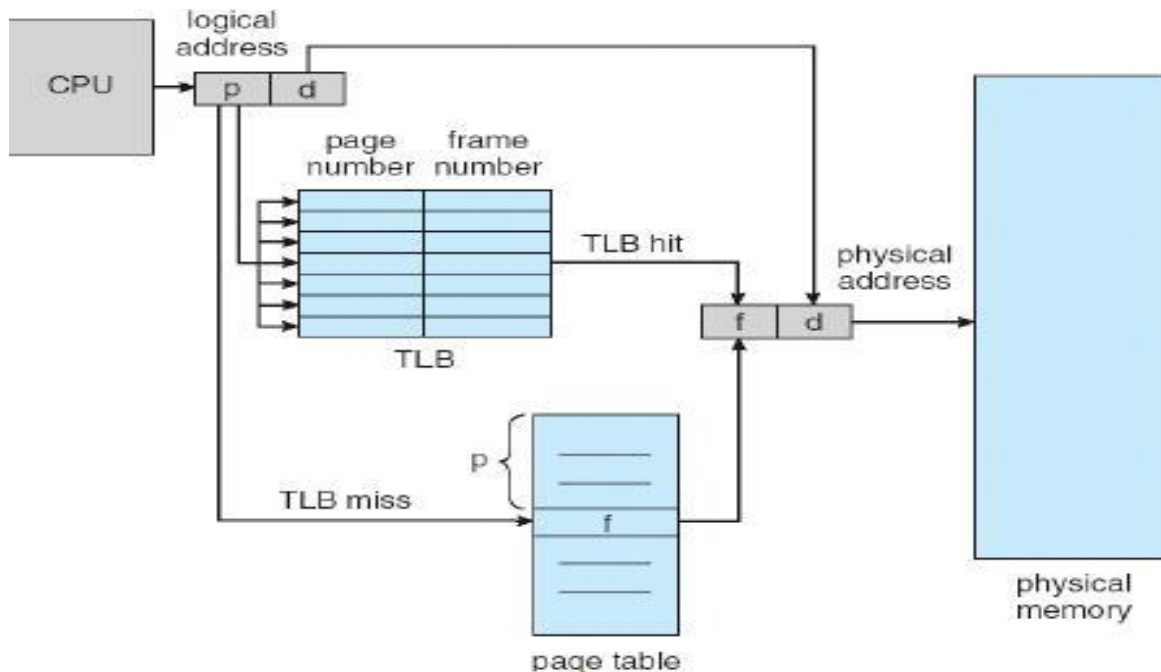


- **Page frame number:** menunjukkan physical frame di mana address dimulai
- **Present/absent:** apakah page yang dirujuk ada di memory
- **Protection:** operasi (read, write, execute) yang diperbolehkan
- **Modified (dirty bit):** apakah page mengalami modifikasi
 - Apabila ada modifikasi, maka konten memory harus ditulis ulang ke disk saat ada pergantian page (page replacement)
- **Referenced:** apakah page sedang dirujuk (memory reference) oleh program
- **Caching disabled:** untuk page yang dipetakan ke I/O device register

Tantangan untuk melakukan Paging yang efisien

- **Pemetaan dari virtual address ke physical address harus cepat**
 - **Gunakan Translation Lookaside Buffer (TLB):**
cache pada MMU yang menyimpan beberapa informasi page table yang sering digunakan → menghemat memory reference
- **Jika virtual address space besar, maka page table juga besar**
 - **Contoh: 32-bit virtual address space, 4KB page (offset = 12 bit), 4 bytes page table entry, maka ukuran page table = $2^{20} \times 4 = 4\text{MB}$. Bagaimana dengan 64-bit virtual address space?**
 - **Gunakan multi-level page table atau inverted page table untuk mengurangi ukuran page table yang disimpan di memory**

Translasi Pengalamatan dengan TLB



- **Hit ratio**: prosentase memory reference yang berhasil ditranslasi oleh TLB
- **Contoh**: jika perlu 20 ns untuk akses TLB dan 100 ns untuk akses memory
 - Apabila TLB hit, maka perlu $20 + 100 = 120$ ns untuk tiap memory reference
 - Apabila TLB miss, maka perlu $20 + 100 + 100 = 220$ ns untuk tiap memory reference
 - Jika hit ratio = 80%, maka effective memory access time = $0.8 \cdot 120 + 0.2 \cdot 220 = 140$ ns

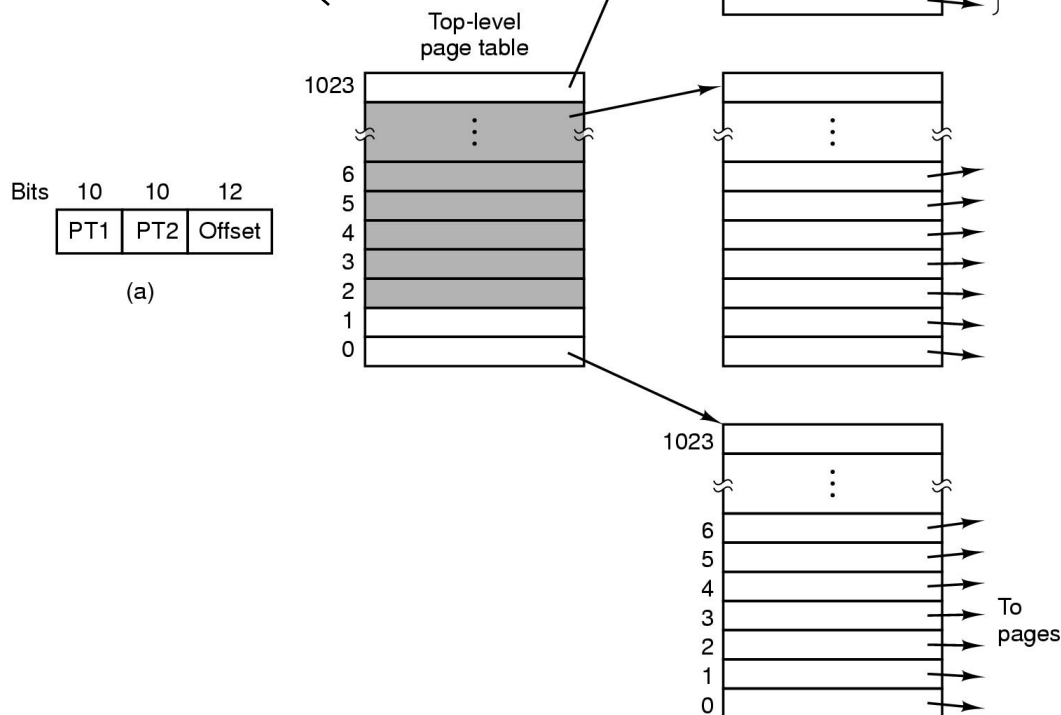
Multilevel Page Tables

Contoh: Two-Level Page Table

asumsi 4bytes per baris

$2^{10} \times 4 = 4\text{KB}$

Harus selalu
di memory



Dapat dibawa ke memory
kalau diperlukan

physical address

page frame #	offset
--------------	--------

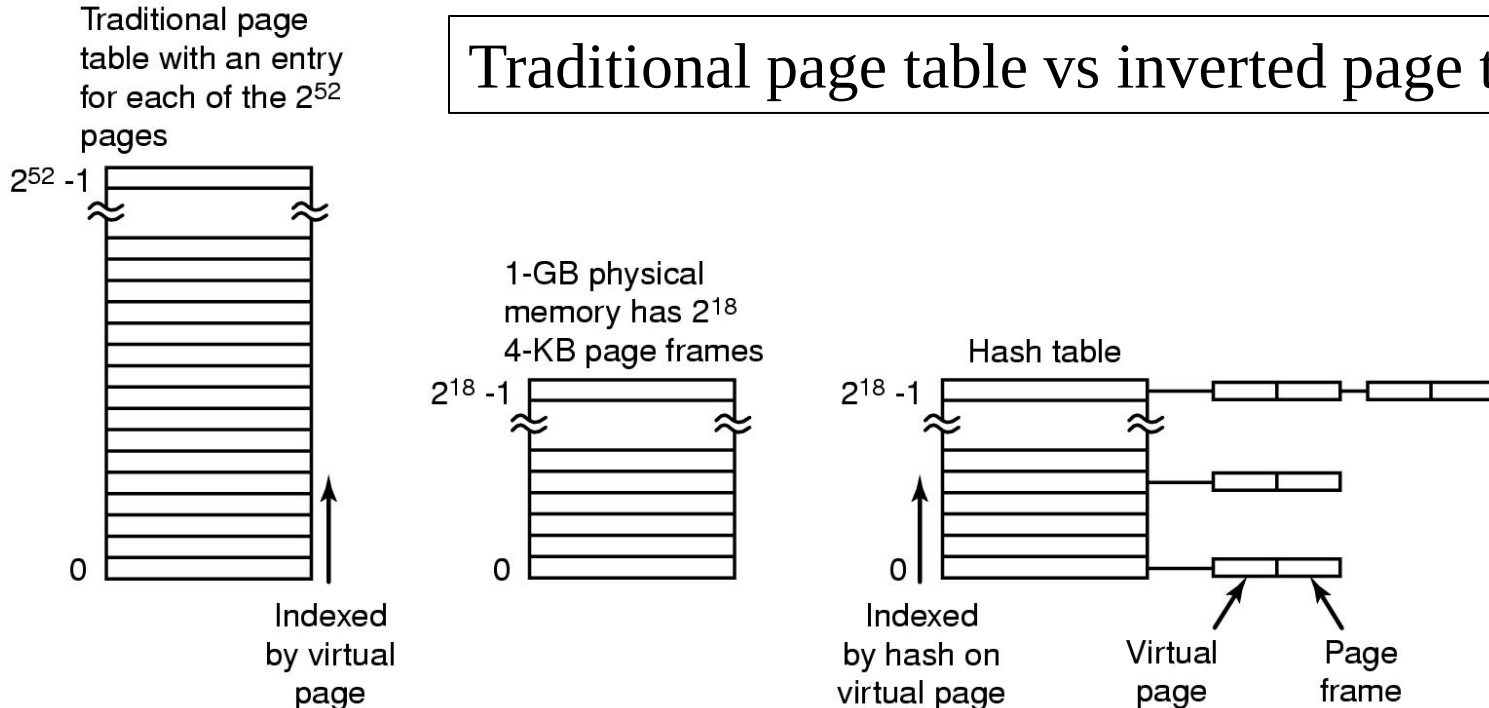
page frame # diperoleh dari entry
pada secondary-level page table
yang dirujuk oleh PT2

Contoh

- 32-bit virtual address 0x00403004 (4,206,596 decimal)
- Dalam binary: 0000 0000 0100 0000 0011 0000 0000 0100
- PT1 = 0000000001 = 1
- PT2 = 0000000011 = 3
- PT1 merujuk pada entry no 1 pada root table, yang kemudian merujuk pada chunk address 4M-8M (4194304 – 8388608)
- Gunakan PT2 yang merujuk pada entry ketiga pada secondary page table yang berarti berada pada page frame dengan 12288 sampai dengan 16382 (absolute address 4,206,592 sampai 4,210,687 decimal)

Inverted Page Tables

- Untuk 64-bit virtual address, multilevel page table masih terlalu besar
- Dengan inverted page table, tiap entry adalah per-page frame di physical memory
- Kekurangan: untuk tiap memory reference, keseluruhan inverted table harus di-search untuk mencari pasangan (virtual,physical)
- Solusi: pasangan (virtual,physical) disusun dalam bentuk hash table dan gunakan TLB untuk mempercepat search.

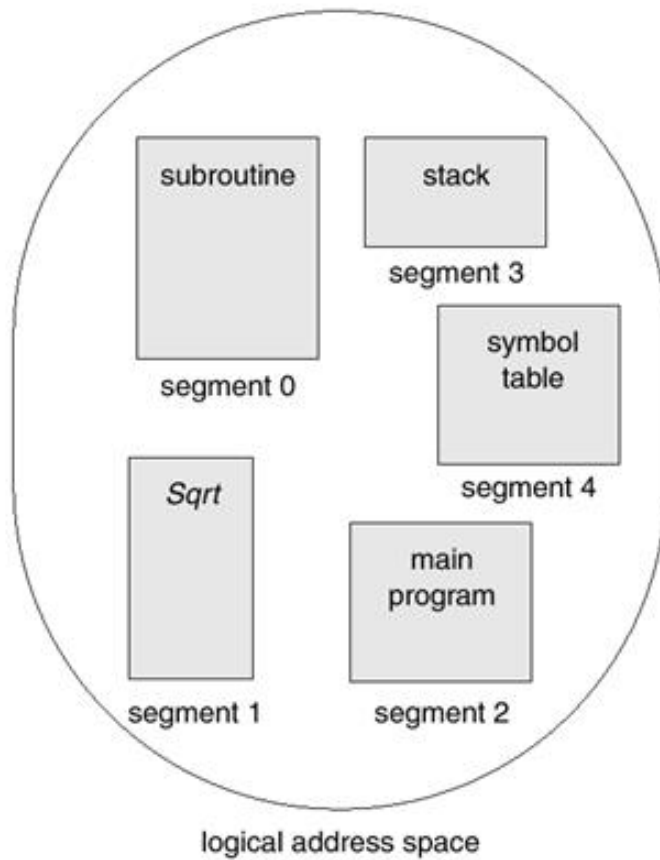


Segmentation (Segmentasi)

Segmentation (Segmentasi)

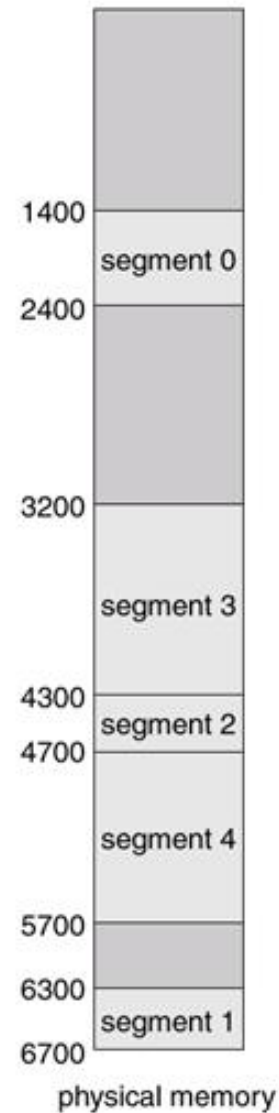
- Teknik untuk alokasi memory berdasarkan segmen logical dari program
 - Program: code segment (main program, function, object, dsb), variables (local, global), stack, symbol table, dsb.
- Compiler membuat referensi untuk mengidentifikasi segment dan offset pada segment sehingga process dapat melihat virtual address nya sebagai segment dan segment offsets. Apabila dikombinasikan dengan Paging, maka segments akan kemudian tersusun sebagai array satu dimensi dari bytes
- Keuntungan segmentation
 - Memudahkan pengguna melihat struktur programnya
 - Lebih mudah dalam mengatur perubahan ukuran stack dan heap secara dinamis
 - Membantu dalam hal sharing dan proteksi penggunaan memory

Contoh

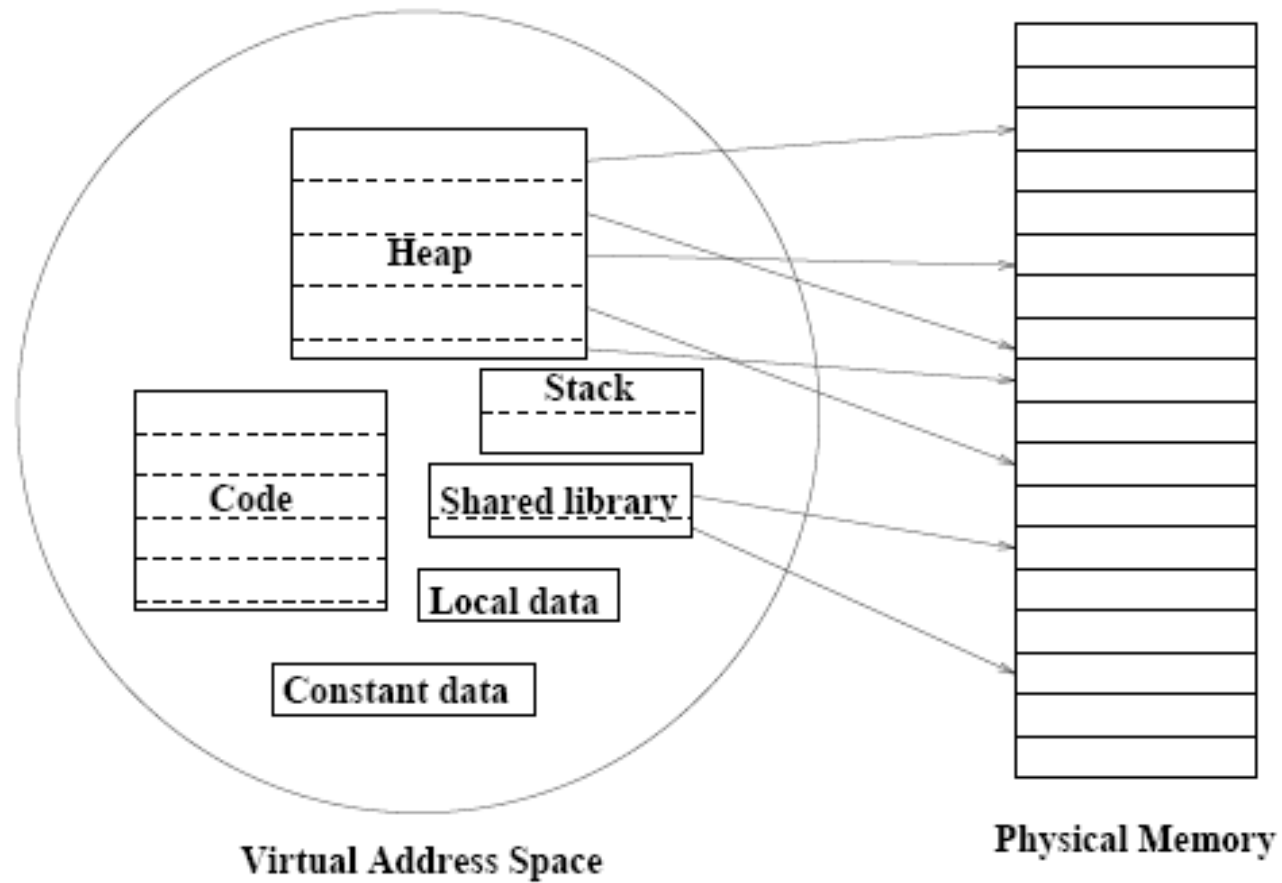


	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table



Segmentation dan Paging



Akhir Kuliah Minggu 9

Terima kasih atas perhatiannya!